A Graph-Attention Surrogate for Solver-Free Multi-Objective Inverse Design of Pixelated Metasurfaces

Saarthak Gupta, M. Gopal Krishna, Anand Kumar, and Debdeep Sarkar

***Abstract*—Optimizing a pixelated metasurface with a full-wave solver in the loop is slow, because every candidate layout needs its own simulation. This paper presents a neural network, a surrogate model, that predicts the complex reflection of a layout in a fraction of a millisecond and replaces the solver. The network reads the 16×16 layout with convolutional layers and graph-attention blocks, which let every pixel exchange information with its neighbours, and then predicts the co-polarized reflection coefficient at every frequency from 2 to 12 GHz. Trained separately on each of the four layout classes of the SUTD-PRCM dataset, on the same number of layouts as the MSDCNN reference models and with one architecture and one set of settings, it has a lower test error than the class-specific MSDCNN models with half or less of their parameters: 48 % lower for shape combinations and 61 % lower for random layouts. A reproduced edge-aware cross-attention surrogate, which encodes the layout with a shallow convolutional network, works on polygons but fails on shape combinations and random layouts, where the proposed surrogate is 2.8–4.1 times more accurate. The surrogate is then used to design in-phase reflectors (artificial magnetic conductors) for X-band made of random pixel layouts. A variational autoencoder (VAE) maps the layouts to continuous vectors, and a multi-objective particle swarm optimizer (MOIPSO) searches these vectors for three goals: a wide in-phase band around 10 GHz, a reflection close to 0 dB in that band, and few fragile pixel features. The paper shows why such a VAE fails for random layouts when its latent space is too small and how to choose its size. A search over $301{,}000$ layouts takes 49 s, and its Pareto front has a hypervolume of 0.455, against 0.295 for the 111,000 CST-simulated layouts of the dataset. CST simulation of nine selected designs confirms the predictions: the magnitude and phase errors are no larger than the surrogate's test errors, and the in-phase band edges agree within 0.2 GHz. Two designs meet the full specification in CST, an in-phase bandwidth of 11.4 % and 13.1 % with less than 0.5 dB in-band loss and only 3 fragile features, which no dataset layout achieves, and five designs have wider in-phase bands (up to 16.3 %) than any dataset layout.**

***Index Terms*—Metasurfaces, inverse design, surrogate model, graph attention, variational autoencoder, particle swarm optimization, artificial magnetic conductor.**

I. Introduction

METASURFACES, thin periodic arrays of sub-wavelength scatterers, control the amplitude, phase and polarization of reflected and transmitted waves [1], [2]. A pixelated metasurface divides the unit cell into an $N \times N$ grid of small metal patches, each either present or absent [3]. Common unit-cell shapes such as square patches, rings, crosses and split rings are all special cases of such a grid, and the grid can be printed directly on a circuit board. This freedom comes at a price: the number of possible layouts is 2^{N^2}, i.e., 2^{256} for a 16×16 grid, far too many to try one by one.

Pixelated layouts are therefore found by optimization. A genetic algorithm [7] or a particle swarm [8], [9] proposes layouts, a full-wave solver computes their response, and the optimizer uses the result to propose better layouts [6], [10]. The optimizer itself is cheap; the solver is not. A swarm of 100 particles run for 100 generations needs 10^4 simulations, and multi-objective problems, whose result is a set of trade-off designs (a Pareto front) rather than one design [11], [12], need even more. The number of simulations one can afford, not the optimizer, then decides how well the design space is explored.

A surrogate model is a neural network trained on simulated data to predict the response of a layout [6]. Once trained, it answers in milliseconds and can replace the solver inside the optimization loop. For unit cells described by a few dimensions (a patch length, a gap) this approach is well established [13]–[16], and generative networks have been used to propose new shapes [17]–[19]. For free-form pixel layouts surrogate modeling is harder, because the response depends on how the pixels connect over the whole cell [20]. The reference study for such layouts is the metasurface design deep convolutional neural network (MSDCNN) of Zhang *et al.* [21], which led to the public SUTD-PRCM dataset [22]. MSDCNN found that no single network worked well for all classes of layouts: a deeper ResNet-34 [23] was needed for shape combinations, and random layouts needed four times more training data and were still predicted least accurately. A convolutional network looks at a small neighbourhood of pixels in each layer and must stack many layers before it can relate distant parts of the layout, which is what determines current paths and resonances.

The proposed surrogate extends a convolutional network in two simple ways. The first is pixel-wise graph attention (PGA) [24], related to graph attention networks [25] and non-local blocks [26]: each pixel learns, for the specific layout, how strongly to use each of its eight neighbours, so that connected metal regions can be followed explicitly. The second, taken

S. Gupta and M. Gopal Krishna contributed equally to this work.

S. Gupta, M. Gopal Krishna, A. Kumar and D. Sarkar are with the Department of Electrical Communication Engineering, Indian Institute of Science, Bengaluru 560012, India. S. Gupta is also with the Department of Applied Mathematics and M. Gopal Krishna with the Department of Electrical Engineering, Delhi Technological University, New Delhi 110042, India (e-mail: saarthakgupta2006@gmail.com; mgopalkrishna0007@gmail.com; anandkumar13@iisc.ac.in; debdeep@iisc.ac.in).

from the edge-aware cross-attention surrogate [27], is a set of learned frequency queries that use attention [28]: each output frequency picks out the parts of the layout that matter for the response at that frequency. The trained surrogate is then used to design in-phase reflectors without a solver in the loop, and the final designs are checked in CST.

The contributions are:

1) **A compact surrogate model with graph attention.** One architecture, trained separately on each of the four SUTD-PRCM layout classes with the same settings and on the same number of layouts as MSDCNN, has a lower test error than the MSDCNN model built for each class, with half or less of its parameters, and than a reproduced edge-aware surrogate whose shallow convolutional encoder fails on the two complex classes.
2) **A VAE design space for random layouts.** A VAE of random layouts is shown to ignore its input when its latent space is too small; a simple rule gives the required size, and a VAE that represents random layouts is trained.
3) **A design method without a solver in the loop.** MOIPSO [59] searches the VAE space and the surrogate scores each layout against three practical goals of an in-phase reflector, all derived from the co-polarized reflection and the layout itself. Such optimizers usually run a full-wave solver in the loop, so that the number of simulations limits the swarm size and the number of generations; here the solver is only used to check the selected designs, and the prediction errors on them are reported.

The paper is organized as follows. Section II describes the dataset and the unit cell. Section III reviews surrogate models of earlier work. Section IV describes the surrogate model and its training, and Section V compares it with MSDCNN and with the edge-aware surrogate. Section VI introduces the VAE, Section VII the inverse design method, and Section VIII the design results. Section IX reports the CST model and the CST check of the designs.

II. Dataset and Unit Cell

A. Unit Cell and Reflection Coefficient

The SUTD-PRCM unit cell [22] is shown in Fig. 1. A 16×16 grid of copper pixels, each 0.5×0.5 mm and 0.018 mm thick, sits inside a 1 mm margin, giving a 10 mm square cell. The pixels are printed on a dielectric substrate with $\varepsilon_r = 2.65$ and $\tan\delta = 0.003$, backed by a 0.18 mm copper ground plane. The cell is repeated periodically in x and y and illuminated by a normally incident plane wave polarized along x. Because of the ground plane, no power is transmitted.

The modeled quantity is the co-polarized reflection coefficient $r_{xx}(f)$, the reflected x-polarized field normalized to the incident field. It is complex: its magnitude tells how much of the wave is reflected with the same polarization, and its phase how the reflected wave is delayed. A layout is written as a 16×16 binary matrix x, with 1 for copper and 0 for no copper.

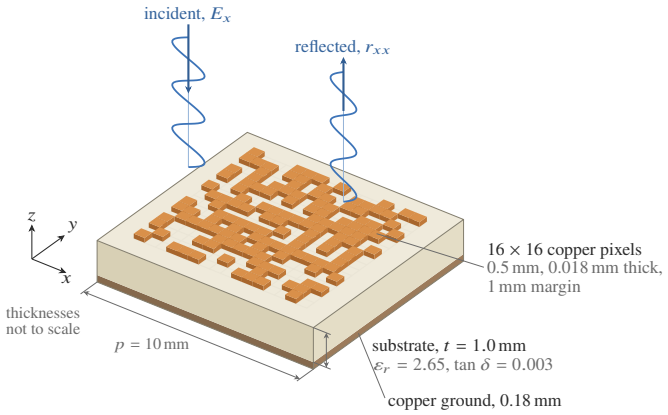


Fig. 1. SUTD-PRCM unit cell with an RDN layout from the test set. An x-polarized plane wave illuminates the cell and is reflected; the co-polarized reflection coefficient r_{xx} is the modeled quantity.

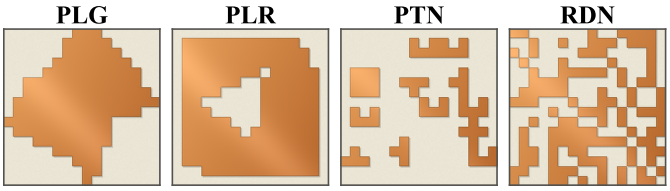


Fig. 2. One layout of each SUTD-PRCM class (copper in orange).

TABLE I
SUTD-PRCM Layout Classes

Class	Layouts	Samples
PLG	filled polygons	30,000
PLR	polygon rings	60,000
PTN	combinations of simple shapes	60,000
RDN	random pixels	111,000

B. Layout Classes

The dataset contains four classes of layouts (Table I, Fig. 2). PLG layouts are filled polygons, PLR layouts polygon rings, PTN layouts combinations of simple shapes, and RDN layouts random pixels in which every pixel is copper with probability one half. RDN contains every possible layout and is the hardest class for a surrogate. It is also the class used for inverse design.

C. Data Preparation

For each layout, the dataset stores r_{xx} from 2 to 12 GHz at 1001 frequencies. As in [27], every tenth frequency is kept, i.e., $K = 101$ frequencies 0.1 GHz apart; the responses are smooth enough at this spacing. For a direct comparison with MSDCNN [21], its protocol is followed: 27,000 training and 3,000 test layouts per class. The layouts of each class are put in a fixed random order. A network is trained on the first 27,000 layouts of this order, and a second RDN network on the first 108,000 RDN layouts; following [21], the networks are named after their training set, e.g., PTN27000 or RDN108000. Every network is tested on the last 3,000 layouts of each class, which none of the networks sees during training. As in [21], no validation set is used.

TABLE II
SURROGATE MODELS FOR METASURFACE AND ANTENNA DESIGN

Work	Design space	Network	Predicted output	Used with
(a) Parametric design spaces				
Liu [13], 2018	thin-film stack (thicknesses)	fully connected, tandem	transmission spectrum	inverse network
Kiarashinejad [29], 2020	nanostructure (dimensions)	autoencoder + fully connected	reflection spectrum	reduced-space inverse design
Niu [30], 2026	1-bit cell (parameters)	tandem + deep Q-network	absorption, polarization conversion	array arrangement
Yu [31], 2026	square-loop FSS (9 parameters)	graph attention + LSTM	$\|S_{21}\|$, 201 frequencies	VAE-PSO
(b) Pixel design spaces: forward surrogates (own datasets)				
Zhu [32], 2021	16×16 coding cell, 8×8 free	Inception-V3, transfer learning	reflection phase, one frequency	phase-to-pattern lookup
Zhang [33], 2022	16×16 isotropic cell, 8×8 free	encoder + fully connected	$\|S_{11}\|$, 0–20 GHz	VAE-PSO
Gupta [34], 2023	12×12 pixel antenna	CNN + tandem inverse	$\|S_{11}\|$	inverse network
Gupta [35], 2024	12×12 pixel antenna	CNN, transfer learning	$\|S_{11}\|$	binary PSO
Chen [36], 2025	10×10 pixel patch	CNN	$\|S_{11}\|$	binary PSO
Aoun [37], 2026	256×256 antenna mask	CNN on VAE latent code	bandwidth, efficiency	reinforcement learning
(c) Pixel design spaces: generative (inverse) models				
Tang [38], 2020	20×20 holes, photonic splitter	conditional VAE	layout from target spectrum	–
Niu [39], 2023	three 128×128 layers	diffusion model	layout from target S-parameters	–
Lyu [40], 2026	multilayer pixelated loop FSS	multichannel conditional GAN	layout from target $\|S_{21}\|$	–
(d) Graph networks for large arrays				
Khoram [41], 2023	arrays of meta-atoms	graph neural network	near field	hologram design
Bao [42], 2025	15×15 meta-atom blocks	graph attention network	near field	metalens design
(e) SUTD-PRCM: public free-form 16×16 dataset, four classes, 2–12 GHz				
MSDCNN [21], 2021	PLG, PTN, RDN; one network per class	ResNet18S / ResNet34S	$\|r_{xx}\|$, 32 frequencies	inverse network
NAS [22], [43], 2022	all four classes	searched shallow, wide CNN	resonance locations, $\|r_{xx}\|$ curve	–
Edge-aware [27], 2026	PLR only	shallow CNN + cross-attention + BiGRU	complex r_{xx}, 101 frequencies	–
This work	**all four classes, one architecture**	**PGA-ResNet + cross-attention + BiGRU**	**complex r_{xx}, 101 frequencies**	**VAE-MOIPSO, 3 objectives**

III. SURROGATE MODELS IN EARLIER WORK

Table II lists surrogate models used for metasurface and antenna design, grouped by design space and by what the network predicts. Reviews of the field are given in [6], [44], [45].

Parametric design spaces (Table II(a)). When the structure is described by a few dimensions, a fully connected network maps them to a spectrum. It is used inside an optimizer [31], paired with an inverse network [13], [30], or combined with dimensionality reduction [29]. A few thousand simulations suffice, but the shapes are fixed in advance.

Pixel design spaces (Table II(b)–(d)). When the cell is a grid of metal pixels, the layout is treated as an image. Forward CNN surrogates predict the reflection phase at one frequency [32] or the magnitude $|S_{11}|$ of 10×10 to 16×16 layouts [33]–[36], and are used with binary or latent-space particle swarms, inverse networks or reinforcement learning [37]. Generative models [38]–[40] produce a layout directly from one target response. Graph networks model the coupling between meta-atoms of large arrays [41], [42]. Each of these studies uses its own dataset of one family of patterns, often reduced by symmetry to 8×8 free pixels, so their accuracies cannot be compared.

SUTD-PRCM (Table II(e)). The dataset of Section II is the only public set of free-form 16×16 layouts with four classes of increasing complexity. Three surrogates have been reported on it, and they are the directly comparable references of this paper. MSDCNN [21] needed one ResNet per class, a deeper one for shape combinations, and had its largest errors on random layouts. A network architecture search [22], [43] found shallow, wide networks best for locating the resonances. The edge-aware surrogate [27] predicts the complex reflection with learned frequency queries, but was tested on one class (PLR) and encodes the layout with a shallow CNN. Both MSDCNN and the edge-aware model are therefore reproduced or quoted under one protocol in Section V.

What this work requires. The inverse design of Sections VII–IX places three demands on the surrogate that no entry of Table II meets together. First, the objectives are the in-phase bandwidth and the in-band loss of a reflector, so the surrogate must predict the complex co-polarized reflection, magnitude and phase, at every frequency of the band; most surrogates above predict only a magnitude or a single-frequency phase. Second, the designs are free-form random layouts, whose response depends on how metal pixels connect across the whole cell; a shallow or purely local encoder cannot follow this, which is why graph attention between neighbouring pixels is added to the convolutional backbone. Third, MOIPSO scores 10^4–10^5 layouts per search against three objectives at once, which needs a fast forward model rather than a generative

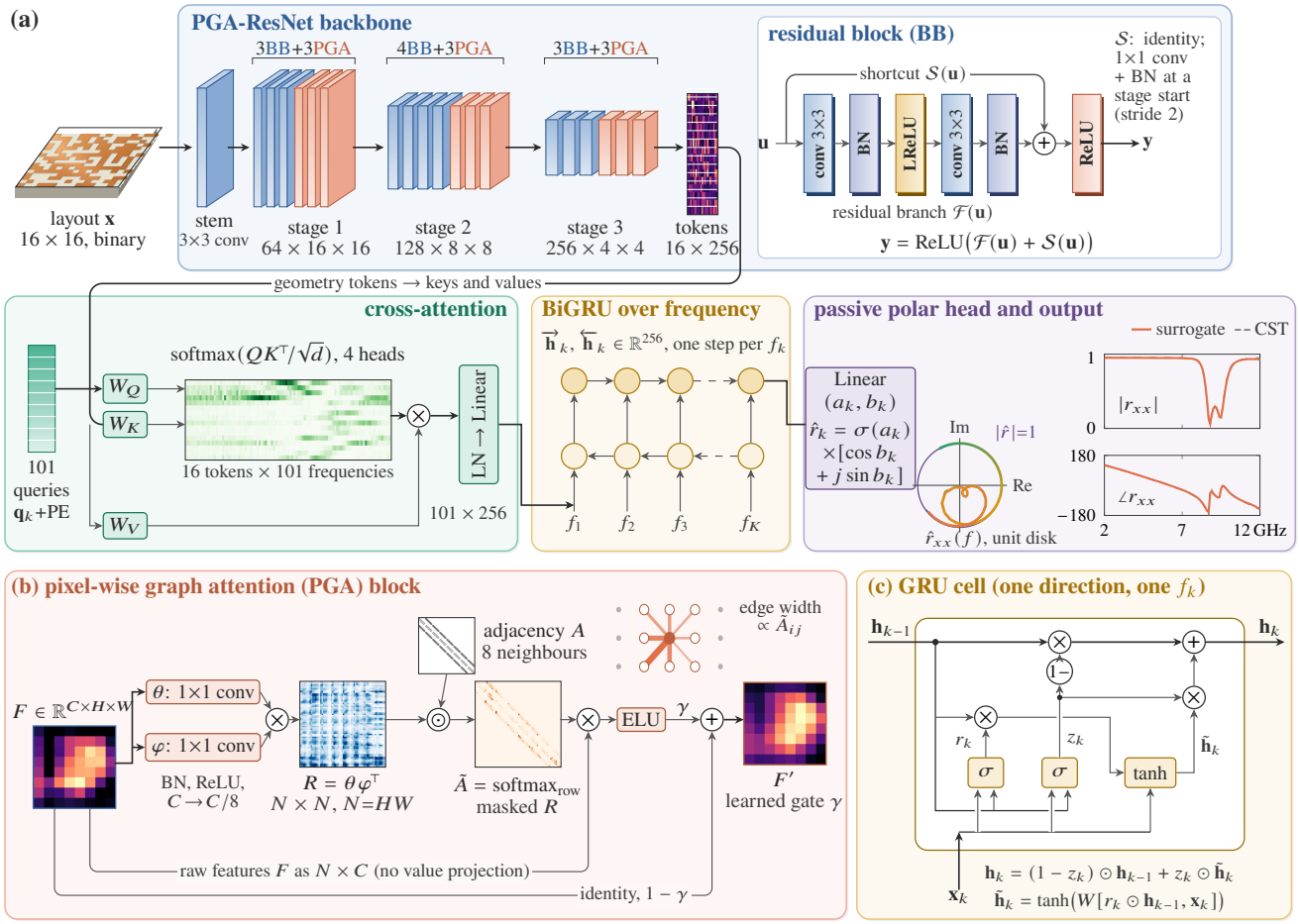


Fig. 3. Surrogate model. (a) Backbone: a stem convolution and three stages of residual blocks (BB, blue) and PGA blocks (orange); the height of each block shows the size of its feature map and its width the number of channels. Top right: one residual block in detail. Below: frequency cross-attention, bidirectional GRU and output layer; the attention weights, the token values and the spectra are those of the trained network RDN108000 for the layout shown. (b) PGA block, with the score, neighbour and attention matrices of one block of the second stage. (c) GRU cell.

model that returns one layout per target. The surrogate of Section IV is built for these three requirements, and one architecture serves all four SUTD-PRCM classes.

IV. Surrogate Model

The surrogate takes a layout x and returns the complex reflection $\hat{S}(f_k)$ at the 101 frequencies. It has three parts (Fig. 3): a convolutional backbone with graph-attention blocks that turns the layout into 16 feature vectors, a frequency cross-attention module and a bidirectional GRU that turn these vectors into one feature vector per frequency, and an output layer that turns each frequency's vector into a complex reflection coefficient.

A. Convolutional Backbone

The layout enters a 3×3 convolution with 64 output channels, followed by batch normalization [46] and a $\tanh$ activation. The stride is 1, so the full 16×16 resolution is kept. Three stages of residual blocks [23] (BB in Fig. 3(a)) follow. A residual block, as in MSDCNN [21], has two paths. The residual branch $\mathcal{F}$ applies a 3×3 convolution, batch normalization, a LeakyReLU activation (slope 0.2), a second 3×3 convolution and a second batch normalization. The shortcut $\mathcal{S}$ passes the block input $\mathbf{u}$ on unchanged. The two are added and a ReLU follows:

$$\mathbf{y} = \mathrm{ReLU}\big(\mathcal{F}(\mathbf{u}) + \mathcal{S}(\mathbf{u})\big). \tag{1}$$

The block therefore only has to learn a correction $\mathcal{F}$ to its input, and during training the gradient can flow back through the shortcut without passing through the convolutions. This is what allows many blocks to be stacked and trained reliably [23]. Each 3×3 convolution combines a pixel with its eight neighbours, so every block widens the region of the layout that a feature can see by two pixels in each direction. The three stages have 3, 4 and 3 blocks with 64, 128 and 256 channels. The first block of the second and third stages uses a stride of 2, which halves the size of the feature map, and its shortcut is then a 1×1 convolution with stride 2 followed by batch normalization, so that the two paths have the same size. The feature maps therefore have 16×16, 8×8 and 4×4 positions. Each stage is followed by three PGA blocks (Section IV-B). After the backbone, channel dropout with probability 0.2 is applied during training, and the 16 positions of the 4×4 map become 16 vectors of 256 numbers, called tokens. Each token describes one region of the layout together with its surroundings.

B. Pixel-Wise Graph Attention

A PGA block [24] lets every position of a feature map collect information from its neighbours with weights that depend on the layout. Let the feature map be F with C channels on an $H \times W$ grid, i.e., $N = HW$ positions with a vector of C numbers each. The block works in five steps (Fig. 3(b)).

1) Neighbour graph. Each position is a node, connected to its eight neighbours: $A_{ij} = 1$ if position j is one of the eight neighbours of position i, and 0 otherwise. A depends only on the grid size and is computed once.

2) Scores. Two 1×1 convolutions, θ and φ, each followed by batch normalization and ReLU, reduce the C channels to $C/8$. The score of node j for node i is the product of the two reduced vectors,

$$R_{ij} = \theta(F)_i^{\top}\, \varphi(F)_j. \tag{2}$$

A high score means that the features at position j are relevant to position i for this particular layout.

3) Attention weights. Scores of positions that are not neighbours are removed, and the remaining scores of each node are turned into weights that sum to one (a softmax):

$$\tilde{A}_{ij} = \frac{A_{ij} \exp R_{ij}}{\sum_{j'} A_{ij'} \exp R_{ij'}}. \tag{3}$$

4) Collecting information. Each node receives the weighted sum of its neighbours' features, $\sum_j \tilde{A}_{ij} F_j$, written $\tilde{A}F$. The original features are used here, not a transformed copy: θ and φ only decide how much each neighbour contributes.

5) Mixing with the input. The result passes through an ELU activation and is mixed with the block input through a learned scalar γ,

$$F' = (1-\gamma)\, F + \gamma\, \mathrm{ELU}\big(\tilde{A}F\big). \tag{4}$$

γ starts at 0, so each block initially passes its input through unchanged and the network only uses the neighbour information as far as it helps [49]. After training, $|\gamma|$ is between 0.14 and 0.36 in every block of RDN108000, i.e., all blocks use it.

Why does this help? A convolution applies the same fixed weights to every neighbourhood. In a PGA block, the weight given to a neighbour depends on the features, so the network can, for example, give more weight to a neighbour that continues a metal path than to one across a gap. Metal connectivity decides where surface currents flow and hence where the resonances lie. Since the graph only connects neighbours, a PGA block costs little: the three stages add 65 k parameters to a 5.8 M-parameter network.

C. Frequency Cross-Attention

The next step must turn 16 tokens, which describe regions of the layout, into 101 vectors, one per frequency. Following [27], each frequency f_k has a learned query vector q_k of 256 numbers, to which a fixed sinusoidal code of the frequency index is added so that the queries know their order [28]. Each query collects information from the tokens by attention:

$$\mathrm{Attn}(q_k, T) = \mathrm{softmax}\!\left(\frac{(W^Q q_k)(W^K T)^{\top}}{\sqrt{d}}\right) W^V T, \tag{5}$$

where T holds the 16 tokens (after layer normalization [47]), W^Q, W^K and W^V are learned matrices and d is the vector length. The softmax gives, for every frequency, a weight for each of the 16 tokens; these weights are shown for the example layout in Fig. 3(a). Four such attention "heads" run in parallel and their outputs are combined. A layer normalization and a linear layer follow. The cost grows linearly with the number of frequencies.

D. Bidirectional GRU

The 101 frequency vectors are then processed in frequency order by a bidirectional gated recurrent unit (GRU) [48] with 256 units in each direction. A GRU (Fig. 3(c)) keeps a state vector $\mathbf{h}_k$ that it updates from one frequency to the next,

$$\mathbf{h}_k = (1 - z_k) \odot \mathbf{h}_{k-1} + z_k \odot \tanh\big(W[r_k \odot \mathbf{h}_{k-1}, \mathbf{x}_k]\big), \tag{6}$$

where $\mathbf{x}_k$ is the input at frequency f_k and the gates r_k and z_k (sigmoid functions of $\mathbf{h}_{k-1}$ and $\mathbf{x}_k$) decide how much of the old state is kept. One GRU runs from 2 to 12 GHz and the other from 12 to 2 GHz, so the prediction at each frequency knows about the response at lower and higher frequencies. This makes the predicted spectra smooth and keeps resonances consistent.

E. Passive Output Layer

After dropout with probability 0.2, a linear layer maps each frequency's 512 numbers to two numbers (a_k, b_k), which give the reflection coefficient

$$\hat{S}(f_k) = \sigma(a_k)\big[\cos b_k + j \sin b_k\big], \tag{7}$$

where $\sigma(a) = 1/(1 + e^{-a})$ is the sigmoid function, which lies between 0 and 1. The predicted magnitude is $|\hat{S}| = \sigma(a_k)$ and the predicted phase b_k. The magnitude is therefore always below one: the predicted surface never reflects more power than it receives, whatever the layout. This matters because the optimizer will query the network with unusual layouts. The network has 5.83 M parameters, and the same architecture is used for every class.

F. Training Loss

The network is trained with the physics-guided loss of [27], a weighted sum of eight terms,

$$\begin{aligned}\mathcal{L} = {} & \mathcal{L}_{\mathrm{reim}} + 0.5\,\mathcal{L}_{\mathrm{dB}} + 0.3\,\mathcal{L}_{\mathrm{lin}} + 0.3\,\mathcal{L}_{\mathrm{ph}} \\ & + 0.08\,\mathcal{L}_{\mathrm{pass}} + 0.06\,\mathcal{L}_{\mathrm{TV}} \\ & + 0.02\,\mathcal{L}_{\mathrm{curv}} + 0.02\,\mathcal{L}_{\mathrm{KK}},\end{aligned} \tag{8}$$

each averaged over the 101 frequencies and listed in Table III. A single error measure is not enough for reflection spectra: a prediction can have a small average error and still misplace a resonance, get its depth wrong, or produce a response that no physical surface could have. Each term therefore checks one property of the spectrum.

Fidelity terms. The real/imaginary term $\mathcal{L}_{\mathrm{reim}}$ compares the complex values directly and is the main term (weight 1); it makes magnitude and phase right together. The magnitude is

TABLE III
TERMS OF THE TRAINING LOSS (8); $\hat{S}$ IS THE PREDICTION AND S THE DATA, k THE FREQUENCY INDEX

Term	What it measures	Formula	Weight
$\mathcal{L}_{\mathrm{reim}}$	error of real and imaginary parts	$\frac{1}{2K}\sum_k \left[(\mathrm{Re}\,\hat{S}_k - \mathrm{Re}\,S_k)^2 + (\mathrm{Im}\,\hat{S}_k - \mathrm{Im}\,S_k)^2\right]$	1.00
$\mathcal{L}_{\mathrm{dB}}$	magnitude error in decibels	$\frac{1}{K}\sum_k \left(20\log_{10}\lvert\hat{S}_k\rvert - 20\log_{10}\lvert S_k\rvert\right)^2$	0.50
$\mathcal{L}_{\mathrm{lin}}$	magnitude error in linear scale	$\frac{1}{K}\sum_k \left(\lvert\hat{S}_k\rvert - \lvert S_k\rvert\right)^2$	0.30
$\mathcal{L}_{\mathrm{ph}}$	phase error (no 2π jumps)	$\frac{1}{K}\sum_k \left\lvert\hat{S}_k/\lvert\hat{S}_k\rvert - S_k/\lvert S_k\rvert\right\rvert^2$	0.30
$\mathcal{L}_{\mathrm{pass}}$	reflection above one	$\frac{1}{K}\sum_k \max(0, \lvert\hat{S}_k\rvert - 1)^2$	0.08
$\mathcal{L}_{\mathrm{TV}}$	frequency-to-frequency jumps	$\frac{1}{K-1}\sum_k \lvert\hat{S}_{k+1} - \hat{S}_k\rvert$	0.06
$\mathcal{L}_{\mathrm{curv}}$	ripple (curvature)	$\frac{1}{K-2}\sum_k \lvert\hat{S}_{k+1} - 2\hat{S}_k + \hat{S}_{k-1}\rvert^2$	0.02
$\mathcal{L}_{\mathrm{KK}}$	Kramers–Kronig consistency	mean squared difference between $\hat{S}$ and its discrete Hilbert-transform estimate	0.02

also compared on two scales. In decibels ($\mathcal{L}_{\mathrm{dB}}$), a resonance dip from -3 to -20 dB is a large difference, so this term makes the network place resonances at the right frequency and with the right depth. In linear scale ($\mathcal{L}_{\mathrm{lin}}$), the same dip is a small difference, while errors near full reflection ($|S| \approx 1$), where most of the band lies, count fully. Both are needed: a dip that is wrong by 10 dB may differ by only 0.05 in linear magnitude. The phase term $\mathcal{L}_{\mathrm{ph}}$ compares the unit phasors $\hat{S}/|\hat{S}|$ and $S/|S|$, i.e., points on the unit circle. Comparing phase angles directly would give a large error between $+179°$ and $-179°$, which are almost the same phase; the phasor form has no such jumps. The phase matters for any design that relies on the reflection phase, such as the in-phase reflector of Section VII.

Physical-consistency terms. The passivity term $\mathcal{L}_{\mathrm{pass}}$ penalizes $|\hat{S}| > 1$, i.e., a surface that reflects more power than it receives. With the output layer (7) this cannot happen, so the term only acts as a safeguard. The two smoothness terms penalize differences between neighbouring frequencies: $\mathcal{L}_{\mathrm{TV}}$ the first difference, which removes spurious jumps, and $\mathcal{L}_{\mathrm{curv}}$ the second difference, which removes ripple between resonances. Their weights are small, so that real narrow resonances are not smoothed away. The last term enforces causality: the reflection of a passive, causal structure has real and imaginary parts that are linked by the Kramers–Kronig relations [50]. For $S = r + jx$ sampled at angular frequencies ω_k, a discrete form of these relations estimates each part from the other,

$$\hat{r}_k = \sum_{j\neq k} \frac{x_j\,\Delta\omega}{\pi(\omega_j - \omega_k)}, \quad \hat{x}_k = -\sum_{j\neq k} \frac{r_j\,\Delta\omega}{\pi(\omega_j - \omega_k)}, \qquad (9)$$

and $\mathcal{L}_{\mathrm{KK}}$ is the mean squared difference between the predicted real and imaginary parts and these estimates. Because the band is finite (2–12 GHz), the estimates are only approximate, and the term has a small weight; it discourages predictions whose real and imaginary parts are inconsistent with each other.

The weights in (8) are those of [27] and are the same for every class.

G. Training Details

Each network is trained on the training layouts of one class only. Training uses the AdamW optimizer [51] with learning rate 2×10^{-4}, weight decay 10^{-5}, batches of 256 layouts and gradient clipping at 1. The learning rate decreases along a cosine curve to 2×10^{-6} over 300 epochs, and the network after the last epoch is used; the test layouts play no part in training or in the choice of any setting.

The unit cell is symmetric and the wave arrives at normal incidence, so mirroring a layout about the x- or y-axis does not change r_{xx}. This is used in two ways. During training, each layout is mirrored at random, which gives the network more examples for free. At test time, the network predicts all four mirror images of a layout and the four predictions are averaged. Training uses one NVIDIA RTX PRO 4000 GPU and takes about 50 min for 27,000 layouts and 3.3 h for 108,000 layouts.

V. SURROGATE RESULTS AND COMPARISON

A. Comparison With MSDCNN

Table IV has the same form as Table I of MSDCNN [21]: each row is a network trained on one class, each column the class on which it is tested, and each entry the mean-squared error (MSE) of $|r_{xx}|$ on the 3,000 test layouts of that class. The upper rows are the MSDCNN results, the middle rows those of the edge-aware surrogate (Section V-B), and the lower rows those of the proposed surrogate. Each proposed network is trained on the same number of layouts as the MSDCNN network of the same name, and all of them are the same 5.83 M-parameter network, whereas MSDCNN needed a larger network for PTN (ResNet34S, 21.3 M parameters) than for the other classes (ResNet18S, 11.2 M).

On its own class (diagonal entries), the proposed network has the lower error for every class. On PTN, PTN27000 reaches 1.43×10^{-3}, 48 % below the 2.76×10^{-3} of MSDCNN's larger PTNRes34, with 27 % of its parameters. On RDN, RDN27000 reaches 4.33×10^{-3} against 10.97×10^{-3} (61 % lower), and RDN108000 2.32×10^{-3} against 4.24×10^{-3} (45 % lower). On PLG the errors are 0.083×10^{-3} (proposed, 2–12 GHz) and 0.201×10^{-3} (MSDCNN, 9.5–12 GHz only). PLR is not part of Table I of [21]; PLR27000 reaches 0.065×10^{-3}. The mean phase error is 0.9–4.9°.

The off-diagonal entries show how well a network predicts a class it has not seen. As for MSDCNN, the networks trained on filled polygons or rings fail on the other classes (errors above 14×10^{-3}). The networks trained on random layouts behave differently: RDN108000 predicts PLG, PLR and PTN with

TABLE IV
TEST MSE OF $|r_{xx}|$ ($\times 10^{-3}$) OF NETWORKS TRAINED ON ONE CLASS (ROWS) AND TESTED ON EACH CLASS (COLUMNS)

	Network (training set)	Layouts	Params	PLG	PLR	PTN	RDN
MSDCNN	PLG27000 (ResNet18S)	27,000	11.2 M	0.201[a]	–	7.517	17.436
	PTN27000 (ResNet18S)	27,000	11.2 M	16.809	–	4.382	12.409
	PTNRes34 (ResNet34S)	27,000	21.3 M	18.426	–	2.760	17.377
	RDN27000 (ResNet18S)	27,000	11.2 M	9.029	–	5.257	10.971
	RDN108000 (ResNet18S)	108,000	11.2 M	14.721	–	13.631	4.242
Edge-aware	PLG27000	27,000	1.88 M	0.145	13.0	25.7	14.0
	PLR27000	27,000	1.88 M	0.647	0.193	21.1	24.3
	PTN27000	27,000	1.88 M	9.16	15.0	3.96	14.0
	RDN27000	27,000	1.88 M	21.1	13.6	4.89	13.2
	RDN108000	108,000	1.88 M	5.97	7.67	3.62	9.43
Proposed	PLG27000	27,000	5.83 M	**0.083**	20.4	14.4	16.2
	PLR27000	27,000	5.83 M	0.426	**0.065**	19.4	25.3
	PTN27000	27,000	5.83 M	10.5	9.18	1.43	9.58
	RDN27000	27,000	5.83 M	3.63	3.22	1.78	4.33
	RDN108000	108,000	5.83 M	1.32	0.986	**0.736**	**2.32**

MSDCNN values from [21, Table I], parameter counts from [22, Table 2]. Edge-aware: reproduction of [27], trained and tested like the proposed networks. All entries: 3,000 test layouts per class. MSDCNN evaluates 32 frequencies, the other networks 101 frequencies over 2–12 GHz. [a]MSDCNN evaluates PLG over 9.5–12 GHz only. Bold: lowest error in each column.

errors of 1.32×10^{-3}, 0.99×10^{-3} and 0.74×10^{-3}; on PTN this is even lower than the error of PTN27000 on its own class. MSDCNN's RDN108000 has 14.7×10^{-3} and 13.6×10^{-3} on PLG and PTN. Random layouts contain every kind of shape, and with graph attention the network learns from them responses that carry over to the structured classes. RDN108000 is the surrogate used for inverse design in Section VII. Fig. 4 compares the predicted and simulated spectra over the test sets in the form of MSDCNN's Fig. 9.

B. Comparison With the Edge-Aware Surrogate

The edge-aware cross-attention surrogate [27] was evaluated on one class (PLR) only. It has the same frequency cross-attention, bidirectional GRU and polar output layer as the proposed surrogate, but it encodes the layout with a shallow convolutional network: a 3×3 convolution, two blocks of convolution and max pooling, and a final convolution turn the layout into 16 tokens, with 1.88 M parameters in total. To test whether such an encoder is enough for all four classes, the network was reproduced from its description (Fig. 1 and the loss (5) of [27], including the $4\times$ weighting of the band edges) and trained and tested in exactly the same way as the proposed surrogate. Its results form the middle block of Table IV.

On filled polygons and rings it works well, with 0.145×10^{-3} on PLG and 0.193×10^{-3} on PLR. On the two harder classes it fails: PTN27000 reaches 3.96×10^{-3} and RDN27000 13.2×10^{-3}, with a coefficient of determination R^2 of only 0.01 and -0.05. R^2 is the share of the layout-to-layout variation of $|r_{xx}|$ that a model explains: 1 for a perfect model and 0 for one that predicts the same average spectrum for every layout. The edge-aware model therefore barely does better than predicting the mean spectrum of the class. On RDN it is worse than MSDCNN (10.97×10^{-3}), and even 108,000 training layouts lower its error only to 9.43×10^{-3}. The shallow encoder applies only four local convolutions before pooling the layout into tokens; this is enough for a single filled shape, but not to represent how metal regions connect across the cell, which decides the current paths and resonances of shape combinations and random layouts. The proposed surrogate keeps the frequency head and replaces the shallow encoder by the residual backbone with graph attention. With 5.83 M parameters, about three times those of the edge-aware model and half those of ResNet18S, its error is 1.7–3.0 times lower than that of the edge-aware model on PLG and PLR, 2.8–4.1 times lower on PTN and RDN, and lower than that of MSDCNN on every class.

C. Contribution of the Graph Attention

To see how much the PGA blocks contribute, RDN108000 was trained once more with all PGA blocks removed and everything else unchanged. Without PGA blocks the test error increases from 2.32×10^{-3} to 2.37×10^{-3}, i.e., by 2.2 %, and the mean phase error rises from 3.41° to 3.49°. The graph-attention blocks give a small but consistent gain for 1.1 % of the parameters (65,193 of 5.83 M).

D. Speed

The RDN network predicts a layout in about 0.15 ms on the GPU, in batches of 2048 layouts. One CST simulation of the same cell takes about 10 min on the workstation used here, i.e., about 4×10^6 times longer.

VI. A VAE DESIGN SPACE FOR RANDOM LAYOUTS

A. Why a Latent Space

A particle swarm moves particles through a continuous space by adding velocities to positions, whereas a layout is a binary matrix. If the swarm works directly on the 256 pixels, every particle position must be rounded to 0 or 1 before it can be scored, and small moves often change nothing while others flip many pixels at once. A variational autoencoder (VAE) [53] offers a continuous alternative, and VAEs have been used to describe metamaterial shapes before [18]. Its encoder maps a layout to a vector z of d real numbers (the latent vector), and its decoder maps any z back to a layout. The swarm then moves in the space of z, and every point it visits decodes to

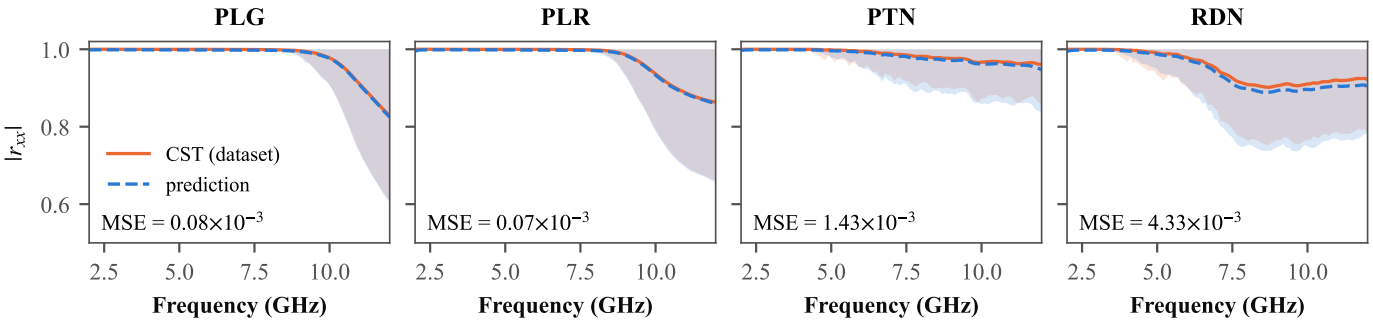


Fig. 4. Predicted and simulated $|r_{xx}|$ over the 3,000 test layouts of each class, as in MSDCNN's Fig. 9: mean (line) and ± 1 standard deviation (band) of the CST data of the dataset and of the predictions of the proposed networks PLG27000, PLR27000, PTN27000 and RDN27000. The number in each panel is the test MSE.

a valid layout. The VAE is trained on RDN layouts, which can represent any layout, and use it together with the RDN surrogate.

B. Architecture and Training

The VAE is a convolutional encoder–decoder (Fig. 6(a)). The encoder has a 3×3 convolution with 32 channels at 16×16, then 3×3 convolutions with stride 2 to 64 channels at 8×8 and 128 channels at 4×4, each with batch normalization and LeakyReLU. Two linear layers map the result to the mean $\boldsymbol{\mu}$ and the log-variance $\log \boldsymbol{\sigma}^2$ of z. During training, z is drawn as $z = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ with random $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$. The decoder mirrors the encoder: a linear layer to $128 \times 4 \times 4$, two transposed convolutions back to 16×16, and a final convolution that gives, for every pixel, the probability that it is copper. Pixels with probability above 0.5 become copper.

The VAE is trained with the loss

$$\mathcal{L}_{\mathrm{VAE}} = 2\,\mathrm{BCE} + \beta\,\mathrm{KL}, \tag{10}$$

where the binary cross-entropy (BCE) measures how well the decoded layout matches the input, and the Kullback–Leibler (KL) divergence measures how far the distribution of z is from a standard normal distribution. The KL term keeps the latent space compact and smooth, so that random points and points between two layouts also decode to meaningful layouts. Both terms are averaged, the BCE over the 256 pixels and the KL over the d latent values, and $\beta = 1$ [54]. For the first five epochs β is zero, and it then increases smoothly to 1 over 30 epochs, which helps the VAE learn to reconstruct first [55]. Training uses Adam [52] (learning rate 10^{-3} decreasing to 10^{-5}), batches of 512, random rotations and mirrors of the layouts, and stops when the validation loss has not improved for 30 epochs.

C. Choosing the Size of the Latent Space

For random layouts, the size d of the latent space decides whether the VAE works at all. Every RDN pixel is an independent coin flip, so a layout carries 256 bits of information and cannot be described by fewer. A VAE stores information about the input in z; each unit of information stored in z costs at least one unit of KL and saves at most one unit of BCE [56]. With the averaging in (10), the KL term written per layout has the weight

$$\beta_{\mathrm{eff}} = \beta\,\frac{N}{2d}, \tag{11}$$

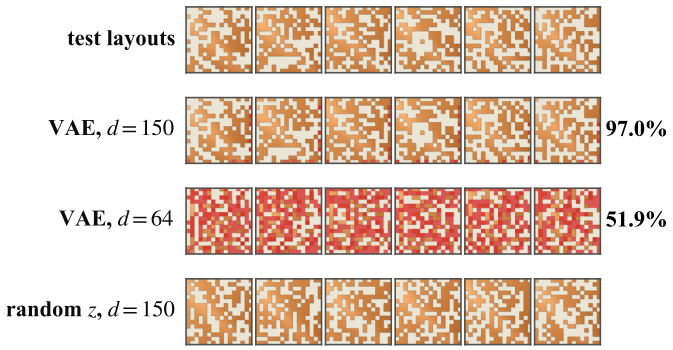


Fig. 5. RDN test layouts, their reconstructions by the VAE with $d = 150$ and with $d = 64$ (wrong pixels in red; right: share of correct pixels), and layouts decoded from random vectors z.

where N is the number of pixels. If $\beta_{\mathrm{eff}} > 1$, storing information costs more than it saves, and the best the VAE can do is to ignore its input and output random layouts, a failure known as posterior collapse [55]. If $\beta_{\mathrm{eff}} < 1$, it pays to store the layout in z.

For 16×16 layouts ($N = 256$) and $\beta = 1$, a latent size of $d = 64$ gives $\beta_{\mathrm{eff}} = 2$. Such a VAE indeed ignores its input: only 50 % of the pixels are reconstructed correctly, which is chance, and none of its 64 latent values carries information. The latent size must therefore exceed $d = 128$; $d = 150$ is used, which gives $\beta_{\mathrm{eff}} = 0.85$ and leaves some margin.

D. VAE Quality

With $d = 150$, the VAE reconstructs 97.4 % of the pixels of the test layouts correctly, and 149 of its 150 latent values carry information. Layouts decoded from random vectors $z \sim \mathcal{N}(0, I)$ look like RDN layouts: they are random pixel patterns with 53 % copper, against 50 % in the dataset. Decoded layouts therefore lie in the region where the surrogate was trained. Fig. 5 shows test layouts, their reconstructions by the two VAEs, and layouts decoded from random vectors.

VII. Inverse Design Method

A. Design Goal: In-Phase Reflector

A metal ground plane reflects a wave with a phase of 180°: the reflected field cancels the incident one at the surface, so an antenna placed close to a metal plane radiates poorly. An artificial magnetic conductor (AMC), or high-impedance surface,

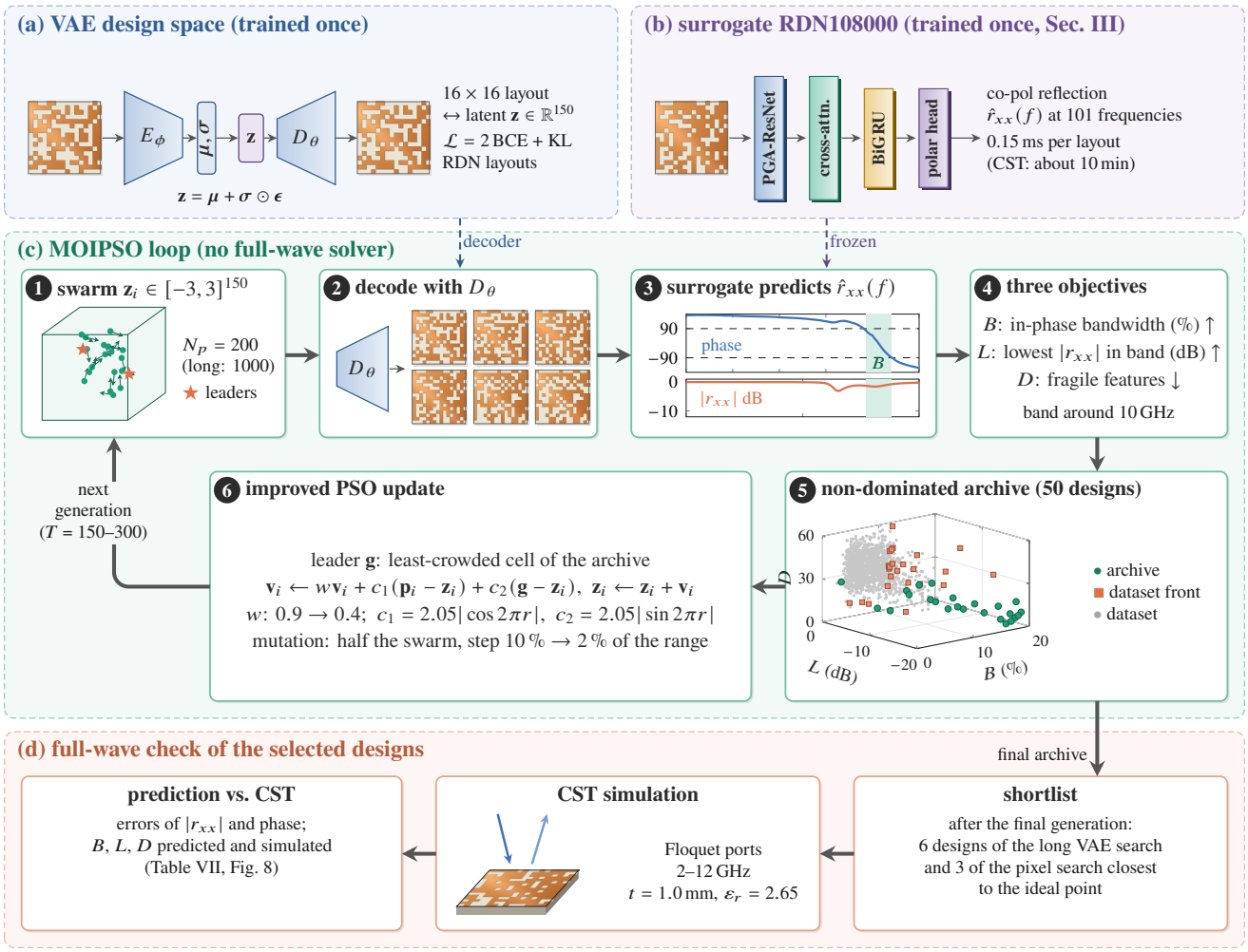


Fig. 6. Design method. (a) VAE of RDN layouts and (b) surrogate RDN108000, both trained once. (c) One generation of MOIPSO in the VAE space, steps 1–6: the particles are decoded into layouts, the surrogate predicts their reflection, the three objectives are computed, the archive (drawn over the RDN dataset) is updated, and the PSO update moves the particles to the next generation. (d) CST check of the designs selected from the final archive.

reflects instead with a phase near 0° (in phase), which allows low-profile antennas to be placed directly above it [4], [5]. The usable band of an AMC is commonly taken as the band in which the reflection phase lies between −90° and +90° [4]. For a thin ground-backed cell like this one, the bandwidth is limited by the substrate thickness, and achieving a wide in-phase band without losses is the main design challenge. Since this goal depends only on the co-polarized reflection, it suits the co-polarized surrogate directly. The design targets X-band, with the in-phase band around $f_0 = 10\,\text{GHz}$.

B. Phase Reference

The phase stored in the dataset, and computed by the CST model of this work in the same way, refers to the plane of the Floquet port, a distance Δ above the top of the cell. Moving the reference to the cell surface multiplies the reflection coefficient by $e^{j2k\Delta}$, with $k = 2\pi f/c$. Δ was determined by simulating the empty cell (substrate and ground only) in CST and comparing its phase with the analytic reflection coefficient of a grounded dielectric slab, $(Z_{\text{in}} - \eta_0)/(Z_{\text{in}} + \eta_0)$ with $Z_{\text{in}} = j(\eta_0/\sqrt{\varepsilon_r})\tan(\beta t)$: with $\Delta = 10.705\,\text{mm}$ the two agree to within 0.03° on average over 2–12 GHz. The same reference is used for the dataset, the surrogate and the CST results, and all phases below refer to the cell surface.

C. Three Objectives

Each design is scored by three numbers, all with a physical unit:

1) *In-phase bandwidth* B (%), to be maximized: the fractional bandwidth $2(f_{\text{hi}} - f_{\text{lo}})/(f_{\text{hi}} + f_{\text{lo}})$ of the band around $f_0 = 10\,\text{GHz}$ in which the surface reflection phase stays within ±90°. If the phase at f_0 is outside ±90°, $B = 0$.
2) *Lowest reflection in the band* L (dB), to be maximized: the smallest $20\log_{10}|r_{xx}|$ inside that band. A random layout can produce narrow resonances in which power is absorbed or converted to the other polarization; such dips inside the band make the reflector lossy. L close to 0 dB means the band is free of them.
3) *Fragile features* D (count), to be minimized: the number of places in which two copper pixels touch only at a corner, plus the number of isolated single copper pixels. A corner-only contact has zero width: whether it conducts depends on the etching tolerance of the board

and, in simulation, on the meshing. An isolated pixel is easily lost in etching. Both make the fabricated response uncertain. A typical RDN layout has about 38 such features.

For the optimizer, each objective is mapped to a penalty between 0 (ideal) and 1 (poor): $(30 - B)/30$, $-L/20\,\mathrm{dB}$ and $D/60$. A design whose phase at f_0 lies outside $\pm 90^\circ$ receives an additional penalty proportional to the phase error, which guides the swarm towards the in-phase condition. A design is regarded as meeting the specification when $B \geq 10\,\%$, $L \geq -1\,\mathrm{dB}$ and $D \leq 5$.

The three objectives compete. In the RDN dataset, the layouts with the widest in-phase bands around 10 GHz have deep dips inside the band and many fragile features: no dataset layout with $B \geq 10\,\%$ and $L \geq -1\,\mathrm{dB}$ has fewer than 17 fragile features. The result of the optimization is therefore a Pareto front: a set of designs in which none is better than another in all three objectives.

D. Comparing Fronts: The Hypervolume

Design a dominates design b if a is at least as good as b in every objective and better in at least one. A design that no other design dominates is Pareto-optimal, and the set of such designs among all designs found is the Pareto front. Two fronts are hard to compare by eye, because each consists of many designs and one front can be better in one region of the objective space and worse in another. The fronts are therefore compared by the hypervolume (HV) [57], the standard single-number measure of a front in multi-objective optimization. Let $\mathbf{p} = (p_1, p_2, p_3)$ be the three penalties of a design, so that $\mathbf{0}$ is the ideal point and $\mathbf{1} = (1, 1, 1)$ the reference point, the worst value of every objective. Each design of the front P dominates the box between its penalty point and the reference point, and the HV is the volume of the union of these boxes,

$$\mathrm{HV}(P) = \mathrm{vol}\Big(\bigcup_{\mathbf{p} \in P} [p_1, 1] \times [p_2, 1] \times [p_3, 1] \Big). \qquad (12)$$

Because the penalties lie between 0 and 1, the HV lies between 0 and 1. It is 1 only if one design is ideal in all three objectives, and it grows whenever a design is added that is not dominated by the front, whether it moves the front closer to the ideal point or extends it to a new trade-off. A larger HV therefore means a front that is better, wider, or both. Equation (12) is computed by Monte Carlo integration: 200,000 random points are drawn uniformly in the unit cube, and the HV is the fraction of them that is dominated by at least one design of the front. The statistical error of this estimate is below 0.002 (below 0.003 for the progress curves of Fig. 7(b), which use 40,000 points). The fronts of the searches (scored by the surrogate) and the front of the RDN dataset (scored with its CST spectra) are evaluated with the same penalties and the same reference point, so their HVs can be compared directly.

E. MOIPSO in the Latent Space

The improved multi-objective particle swarm optimizer (MOIPSO) of [59] is used with the settings of Table V. Each

TABLE V
MOIPSO SETTINGS

Setting	Value
Search space	VAE latent vector, $z \in [-3, 3]^{150}$
Particles / generations	200 / 150 (standard); 1000 / 300 (long)
Objectives	B (%), L (dB), D (count)
Archive size	50, grid-based leader selection
Inertia ω	0.9 to 0.4, linear
Acceleration c_1, c_2	$2.05\lvert\cos 2\pi\rho_1\rvert$, $2.05\lvert\sin 2\pi\rho_2\rvert$
Mutation	half of the swarm, step 10 % to 2 % of range
Maximum velocity	0.5 per component
Initial positions	random, $z \sim \mathcal{N}(0, I)$

particle i has a position z_i in the latent space, restricted to $[-3, 3]^{150}$, and a velocity v_i. In every generation,

$$v_i \leftarrow \omega v_i + c_1(p_i - z_i) + c_2(g_i - z_i), \qquad z_i \leftarrow z_i + v_i, \quad (13)$$

where p_i is the best position the particle has found itself and g_i a leader taken from the archive described below. The inertia ω decreases linearly from 0.9 to 0.4, so that the swarm first explores and later converges. The factors $c_1 = 2.05\lvert\cos 2\pi\rho_1\rvert$ and $c_2 = 2.05\lvert\sin 2\pi\rho_2\rvert$, with random ρ_1, ρ_2 drawn every generation, balance the pull towards the particle's own best and towards the leader; this is the main change of MOIPSO over the standard swarm.

Archive and leaders. An external archive keeps the best trade-off designs found so far, up to 50. The objective space is divided into a grid of boxes, and leaders are chosen preferably from boxes with few archive members, which keeps the front spread out [58]. When the archive is full, members in crowded regions are removed first [12]. Since many latent vectors decode to the same layout, the archive keeps one entry per layout.

Mutation and bounds. In every generation, half of the particles receive a random Gaussian step, whose size decreases from 10 % to 2 % of the search range over the run, to escape local optima. Particles that leave the box $[-3, 3]^{150}$ are reflected back, and each velocity component is limited to 0.5.

Personal best. A particle's best position p_i is replaced when the new position is better in all objectives, and with probability 0.5 when neither is better.

F. The Complete Loop

One generation of the design loop (Fig. 6(c), steps 2–6) has five steps:

1) the decoder turns the latent vectors of all particles into binary layouts;
2) the surrogate predicts r_{xx} of all layouts in one batch;
3) the three objectives are computed for each layout;
4) the archive and the personal bests are updated;
5) leaders are chosen and the velocities and positions are updated.

No full-wave simulation is needed inside the loop. After the last generation, the archive is the predicted Pareto front, and the designs closest to the ideal (smallest sum of the three penalties) are simulated in CST (Section IX).

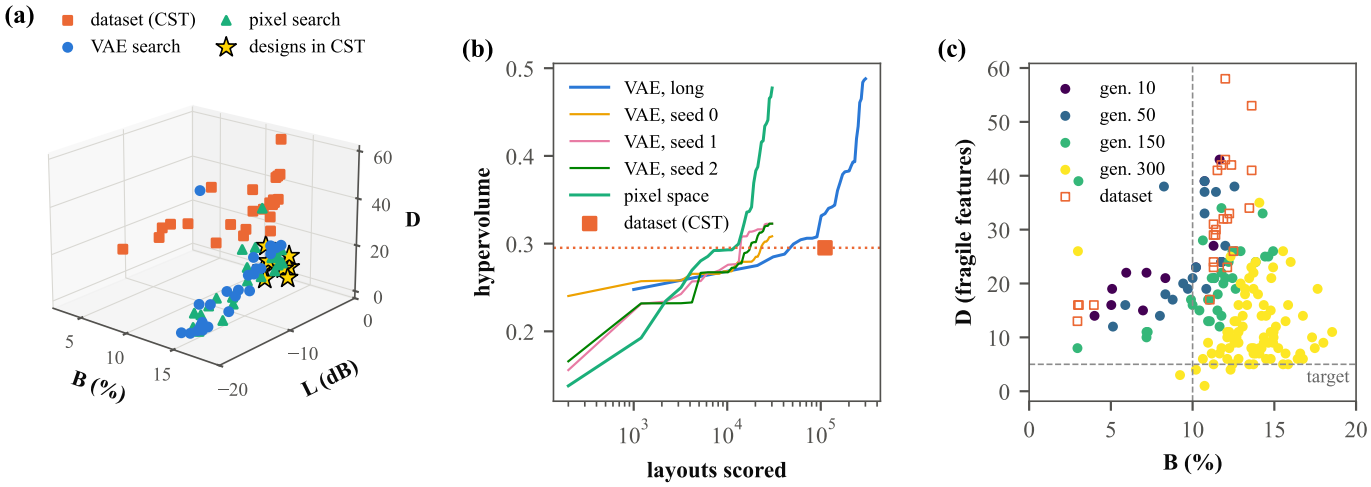


Fig. 7. MOIPSO results. (a) Pareto fronts of the long VAE-space search and of the pixel-space search (predicted) and of the RDN dataset (CST) in the three objectives; stars: CST results of the simulated designs of Table VII. Dips below −20 dB are drawn at −20 dB. (b) Hypervolume of the best front among all layouts scored so far, against the number of layouts scored; square and dotted line: the 111,000 CST-simulated dataset layouts. (c) Front of the long search after 10, 50, 150 and 300 generations, shown as in-phase bandwidth against fragile features; dashed: $B = 10\,\%$ and $D = 5$.

VIII. Design Results

A. Search Cost

One search with 200 particles over 150 generations scores 30,200 layouts in 7–9 s on one GPU, including decoding; a longer search with 1000 particles over 300 generations scores 301,000 layouts in 49 s (Table VI). At about 10 min per CST simulation, the long search would take almost six years of simulation. All searches use the surrogate RDN108000. The standard search was run three times with different random seeds, once with the longer budget, and once directly in pixel space for comparison.

B. Comparison With the Dataset

The 111,000 RDN layouts of the dataset, with their CST spectra, are the baseline: they show what the best existing random layout achieves for the same objectives. No dataset layout meets all three specifications at once; the outcome of each search is a set of trade-offs (Fig. 7).

Table VI compares the searches by the hypervolume (12) of their final archives. The long search raises it from 0.295 for the dataset to 0.455, and the best in-phase bandwidth from 13.6 % to 18.5 %. Its front also reaches far fewer fragile features: it contains a design with 10.7 % in-phase bandwidth and a single fragile feature, and designs with 15–17 % bandwidth and 5–8 fragile features, whereas every dataset layout with at least 10 % bandwidth has 17 or more. Among the layouts it scored, 9,077 have $B \geq 10\,\%$ and $L \geq -1$ dB, the best of them with only 4 fragile features; its final archive, which keeps 50 designs spread over the whole front, favours wider bands with deeper dips and holds none of them. The pixel-space search keeps one such design in its archive, which according to the surrogate meets the full specification: an in-phase band from 9.4 to 10.6 GHz ($B = 12.0\,\%$), a lowest reflection of −0.70 dB in the band and 3 fragile features. In the dataset, the layouts with $B \geq 10\,\%$ and $L \geq -1$ dB have 17 or more fragile features. The standard VAE searches reach about the level of the dataset (HV 0.29–0.32). All search values are predicted by the surrogate; the CST check follows in Section IX.

TABLE VI
Search Results for the X-Band In-Phase Reflector

Search	Layouts	Time	HV	Best B	Fewest D^a
RDN dataset (CST)	111,000	–	0.295	13.6 %	17
VAE space (3 runs)	30,200	7–9 s	0.29–0.32	12.7–13.1 %	12–14
VAE space, long	301,000	49 s	0.455	**18.5 %**	4
Pixel space	30,200	7 s	**0.464**	17.8 %	**3**

HV: hypervolume (12) (ideal design: 1). The dataset front scored by the surrogate gives 0.282. [a]Fewest fragile features among all layouts with $B \geq 10\,\%$ and $L \geq -1$ dB that the search scored (dataset: among all its layouts). Search values are predicted by the surrogate.

C. Progress Over the Iterations

Fig. 7(b) shows how fast each search improves on the dataset. Its horizontal axis is the number of layouts scored by the surrogate; the dataset appears as a single point, 111,000 layouts each simulated in CST, with a hypervolume of 0.295. The pixel-space search passes this value after about 12,000 layouts, the standard VAE searches after 14,000–24,000 and the long search after about 50,000, i.e., after scoring 11–45 % as many layouts as the dataset contains, at a fraction of a second instead of ten minutes per layout. The standard VAE searches then level off at 0.31–0.32. The pixel-space search and the long VAE search keep improving until their last generation, to 0.48 and 0.49. (These curves count every layout scored, so their final values lie slightly above those of the 50-design archives in Table VI.) Fig. 7(c) shows how the front of the long search moves. After 10 generations the in-phase bandwidth is at most 11.8 %, and designs with $B \geq 10\,\%$ have 23 or more fragile features, as the dataset layouts do. After 50 and 150 generations this number falls to 19 and 12, and after 300 generations the front reaches 18.5 % bandwidth and holds several designs with $B \geq 10\,\%$ and 5 or fewer fragile features: the swarm moves step by step towards the target corner of the plot, and in 3-D (Fig. 7(a)) both searched fronts lie below the dataset front in the fragile-feature direction.

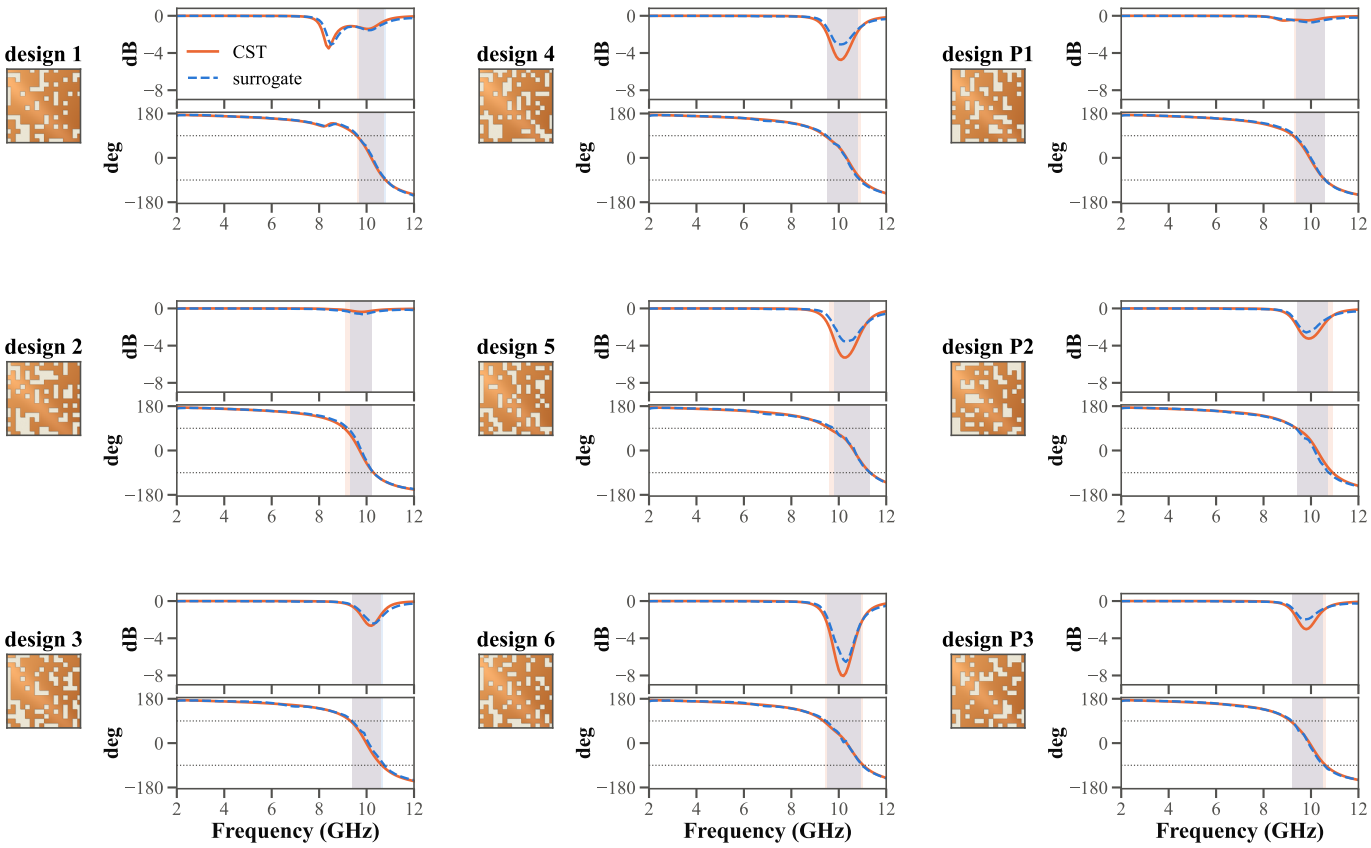


Fig. 8. All nine designs of Table VII, designs 1–6 of the long VAE search and P1–P3 of the pixel-space search: layout, $|r_{xx}|$ and surface reflection phase, predicted by the surrogate (dashed) and simulated in CST (solid). Shaded: in-phase band (blue: predicted, orange: CST). Dotted lines: $\pm 90°$.

D. VAE Space or Pixel Space

For comparison, the swarm was also run directly on the 256 pixels, rounding every particle position to 0 or 1 before scoring. With the same number of evaluations as a standard VAE search, it reaches a larger hypervolume (0.464 against 0.29–0.32), slightly above the long VAE search with ten times the budget, and it is the only search whose final archive holds a design predicted to meet the full specification. For random layouts, therefore, the VAE does not make the search easier. As shown in Section VI, its latent space must be about as large as the layout itself, and since many latent vectors decode to the same layout, part of the swarm's moves do not change the layout at all. What makes a search over 10^4–10^5 layouts affordable is the surrogate, in either space.

IX. CST Verification

A. CST Model of the Unit Cell

To simulate the designs, the unit cell was rebuilt in CST Studio Suite with the published geometry and materials: 16×16 copper pixels of $0.5 \times 0.5 \times 0.018$ mm with a 1 mm margin, $\varepsilon_r = 2.65$, $\tan\delta = 0.003$, a 0.18 mm copper ground, unit-cell boundaries in x and y, Floquet ports for the two polarizations, and the frequency-domain solver from 2 to 12 GHz. The substrate is 1.0 mm thick. The phase reference of the model, 10.705 mm above the cell, was determined from the empty cell (Section VII). A difference between surrogate and CST for a new design therefore shows the error of the surrogate on that design.

TABLE VII
PREDICTION VS. CST FOR THE SELECTED DESIGNS

	Error		B (%)		L (dB)		
Design	MSE	Phase	Pred.	CST	Pred.	CST	D
VAE space, long search							
1	0.56	2.3°	10.7	10.8	−1.55	−1.41	1
2	0.10	3.0°	9.2	**11.4**	−0.62	**−0.38**	**3**
3	0.28	3.9°	12.9	12.0	−2.41	−2.66	14
4	1.18	2.3°	12.8	13.7	−3.08	−4.75	12
5	1.76	2.8°	14.2	16.3	−3.53	−5.28	18
6	0.91	2.4°	13.7	15.7	−6.55	−8.06	11
Pixel space							
P1	0.12	2.3°	12.0	**13.1**	−0.70	**−0.49**	**3**
P2	0.45	4.0°	12.9	14.8	−2.59	−3.25	4
P3	0.59	2.3°	13.2	14.1	−1.98	−3.03	7
RDN test set	2.4	3.9°	–	–	–	–	–

MSE of $|r_{xx}|$ ($\times 10^{-3}$) and mean phase difference over 2–12 GHz between prediction and CST. D: fragile features, a property of the layout. Bold: designs that meet the full specification ($B \geq 10\,\%$, $L \geq -1$ dB, $D \leq 5$) in CST.

B. Verification of the Designs

The optimizer looks for layouts with the best predicted response. Among hundreds of thousands of layouts it may also find some whose response the surrogate predicts too optimistically, so the test error alone does not tell how accurate the predictions for the selected designs are. The six designs of the long VAE search and the three designs of the pixel-space search closest to the ideal (smallest sum of the three penalties) were simulated in CST, one simulation each of about 10 min. Table VII reports, for each design, the MSE of $|r_{xx}|$ and the

mean phase difference between prediction and CST over 2–12 GHz, and the three objectives as predicted and as simulated. Fig. 8 compares the spectra of all nine designs.

Magnitude. The MSE of $|r_{xx}|$ between prediction and CST lies between 0.10×10^{-3} and 1.8×10^{-3}, with a mean of 0.66×10^{-3}, for all nine designs below the surrogate's test error on RDN (2.4×10^{-3}). The optimizer has therefore not driven the designs into a region where the surrogate is unreliable. Where the surrogate predicts no strong dip in the band (designs 1, 2 and P1), the simulated in-band loss is even lower than predicted, e.g., -0.38 instead of -0.62 dB for design 2. Where it predicts a dip (designs 3–6, P2 and P3), the dip is at the right frequency but 0.3–1.8 dB deeper in CST (Fig. 8, designs 4–6).

Phase. The mean phase difference is 2.3–4.0°, the same as the test error of 3.9°. Even where the reflection phase falls steeply through $\pm90°$, the edges of the in-phase band are predicted within 0.2 GHz (Table VII and Fig. 8).

Objectives. The simulated in-phase bandwidths are within 1 percentage point of the prediction for designs 1, 3, 4 and P3 and 1.1–2.2 points wider for the others; every simulated design has at least 10.8 % in-phase bandwidth. Two designs meet the full specification in CST: design 2 (11.4 %, -0.38 dB, 3 fragile features) and design P1 (13.1 %, -0.49 dB, 3 fragile features). No layout of the dataset meets it, and no dataset layout has both a wider in-phase band and a lower in-band loss than either of them. Designs 4, 5, 6, P2 and P3 have simulated in-phase bands of 13.7–16.3 %, wider than that of any dataset layout (at most 13.6 %), at the price of deeper dips in the band. Design 1 (10.8 %, -1.41 dB) has a single fragile feature, whereas the 43 dataset layouts that are at least as good in bandwidth and loss have 17 or more.

X. Discussion

Accuracy where the optimizer searches. The optimizer selects the best predictions among many thousands, so the selected designs are where an over-optimistic surrogate would show first. In CST the magnitude and phase errors of the selected designs are no larger than the test errors, and the in-phase band edges are predicted within 0.2 GHz, so the surrogate can be trusted for the bandwidth objective. The one systematic difference is the depth of predicted in-band dips: a narrow resonance that the surrogate places correctly is up to 1.8 dB deeper in CST, because the training loss averages over all 101 frequencies and a sharp minimum covers only a few of them. A larger weight of the decibel term near resonances, or a margin on the in-band loss objective, would address this.

Scope. The surrogate is trained for normal incidence and one substrate. Other substrates, cell sizes or incidence angles need new training data, but no change to the architecture or the training settings.

Design space. For random layouts the VAE gives no compression of the search space, because the layouts cannot be compressed, and a swarm working directly on the pixels did at least as well. The VAE's value lies in structured design spaces, where the latent space is much smaller than the layout.

XI. Conclusion

This paper presented a surrogate model for pixelated metasurfaces that combines a convolutional backbone with pixel-wise graph attention, frequency cross-attention, a bidirectional GRU and a passive output layer. Trained separately on each of the four SUTD-PRCM layout classes with one architecture, one set of settings and the same number of layouts as MSDCNN, it has a lower error than the class-specific MSDCNN models with half or less of their parameters (48 % lower on shape combinations, 61 % lower on random layouts with 27,000 and 45 % with 108,000 training layouts), and the network trained on random layouts also predicts the structured classes. A reproduced edge-aware surrogate with a shallow convolutional encoder, trained in the same way, matches it only on polygons and fails on shape combinations and random layouts, which shows that the backbone, not the frequency head, decides whether one architecture suits all classes. It was shown that a VAE of random layouts needs an effective KL weight below one, and sized it accordingly. Used in place of the solver, the surrogate lets a multi-objective particle swarm explore 301,000 layouts in 49 s and find X-band in-phase reflectors whose Pareto front dominates that of the 111,000-layout dataset. CST simulation of nine selected designs confirms the predictions: the magnitude and phase errors are no larger than the surrogate's test errors and the in-phase band edges agree within 0.2 GHz. Two designs meet the full specification in CST, with 11.4 % and 13.1 % in-phase bandwidth, less than 0.5 dB in-band loss and 3 fragile features, which no dataset layout achieves, and five designs have wider in-phase bands (up to 16.3 %) than any dataset layout.

References

[1] C. L. Holloway, E. F. Kuester, J. A. Gordon, J. O'Hara, J. Booth, and D. R. Smith, "An overview of the theory and applications of metasurfaces: The two-dimensional equivalents of metamaterials," *IEEE Antennas Propag. Mag.*, vol. 54, no. 2, pp. 10–35, Apr. 2012.

[2] S. B. Glybovski, S. A. Tretyakov, P. A. Belov, Y. S. Kivshar, and C. R. Simovski, "Metasurfaces: From microwaves to visible," *Phys. Rep.*, vol. 634, pp. 1–72, 2016.

[3] T. J. Cui, M. Q. Qi, X. Wan, J. Zhao, and Q. Cheng, "Coding metamaterials, digital metamaterials and programmable metamaterials," *Light Sci. Appl.*, vol. 3, Art. no. e218, 2014.

[4] D. Sievenpiper, L. Zhang, R. F. J. Broas, N. G. Alexopolous, and E. Yablonovitch, "High-impedance electromagnetic surfaces with a forbidden frequency band," *IEEE Trans. Microw. Theory Techn.*, vol. 47, no. 11, pp. 2059–2074, Nov. 1999.

[5] F. Yang and Y. Rahmat-Samii, "Reflection phase characterizations of the EBG ground plane for low profile wire antenna applications," *IEEE Trans. Antennas Propag.*, vol. 51, no. 10, pp. 2691–2703, Oct. 2003.

[6] O. Khatib, S. Ren, J. Malof, and W. J. Padilla, "Deep learning the electromagnetic properties of metamaterials—A comprehensive review," *Adv. Funct. Mater.*, vol. 31, no. 31, Art. no. 2101748, 2021.

[7] S. Jafar-Zanjani, S. Inampudi, and H. Mosallaei, "Adaptive genetic algorithm for optical metasurfaces design," *Sci. Rep.*, vol. 8, Art. no. 11040, 2018.

[8] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Netw. (ICNN)*, 1995, pp. 1942–1948.

[9] J. Robinson and Y. Rahmat-Samii, "Particle swarm optimization in electromagnetics," *IEEE Trans. Antennas Propag.*, vol. 52, no. 2, pp. 397–407, Feb. 2004.

[10] Q. Zhang, X. Wan, S. Liu, J. Y. Yin, L. Zhang, and T. J. Cui, "Shaping electromagnetic waves using software-automatically-designed metasurfaces," *Sci. Rep.*, vol. 7, Art. no. 3588, 2017.

[11] C. A. Coello Coello, G. B. Lamont, and D. A. Van Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY, USA: Springer, 2007.

[12] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[13] D. Liu, Y. Tan, E. Khoram, and Z. Yu, "Training deep neural networks for the inverse design of nanophotonic structures," *ACS Photon.*, vol. 5, no. 4, pp. 1365–1369, 2018.

[14] I. Malkiel, M. Mrejen, A. Nagler, U. Arieli, L. Wolf, and H. Suchowski, "Plasmonic nanostructure design and characterization via deep learning," *Light Sci. Appl.*, vol. 7, Art. no. 60, 2018.

[15] C. C. Nadell, B. Huang, J. M. Malof, and W. J. Padilla, "Deep learning for accelerated all-dielectric metasurface design," *Opt. Express*, vol. 27, no. 20, pp. 27523–27535, 2019.

[16] S. An, C. Fowler, B. Zheng, M. Y. Shalaginov, H. Tang, H. Li, L. Zhou, J. Ding, A. M. Agarwal, C. Rivero-Baleine, K. A. Richardson, T. Gu, J. Hu, and H. Zhang, "A deep learning approach for objective-driven all-dielectric metasurface design," *ACS Photon.*, vol. 6, no. 12, pp. 3196–3207, 2019.

[17] Z. Liu, D. Zhu, S. P. Rodrigues, K.-T. Lee, and W. Cai, "Generative model for the inverse design of metasurfaces," *Nano Lett.*, vol. 18, no. 10, pp. 6570–6576, 2018.

[18] W. Ma, F. Cheng, Y. Xu, Q. Wen, and Y. Liu, "Probabilistic representation and inverse design of metamaterials based on a deep generative model with semi-supervised learning," *Adv. Mater.*, vol. 31, no. 35, Art. no. 1901111, 2019.

[19] J. Jiang and J. A. Fan, "Global optimization of dielectric metasurfaces using a physics-driven neural network," *Nano Lett.*, vol. 19, no. 8, pp. 5366–5372, 2019.

[20] T. Qiu, X. Shi, J. Wang, Y. Li, S. Qu, Q. Cheng, T. Cui, and S. Sui, "Deep learning: A rapid and efficient route to automatic metasurface design," *Adv. Sci.*, vol. 6, no. 12, Art. no. 1900128, 2019.

[21] T. Zhang, C. Y. Kee, Y. S. Ang, and L. K. Ang, "Deep learning-based design of broadband GHz complex and random metasurfaces," *APL Photon.*, vol. 6, no. 10, Art. no. 106101, 2021.

[22] T. Zhang, Y. S. Ang, E. Li, C. Y. Kee, and L. K. Ang, "SUTD-PRCM dataset and neural architecture search approach for complex metasurface design," *IEEE Antennas Wireless Propag. Lett.*, vol. 21, no. 7, pp. 1317–1321, Jul. 2022.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[24] W. Zhang, Q. Ding, J. Hu, Y. Ma, and M. Lu, "Pixel-wise graph attention networks for person re-identification," 2023, *arXiv:2307.09183*.

[25] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[26] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 7794–7803.

[27] C. Zhang, W. Zhu, L. Wu, and F. Long, "Edge-aware cross-attention surrogate for phase-stable metasurface reflection," *IEEE Antennas Wireless Propag. Lett.*, vol. 25, no. 6, pp. 2461–2465, Jun. 2026.

[28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[29] Y. Kiarashinejad, S. Abdollahramezani, and A. Adibi, "Deep learning approach based on dimensionality reduction for designing electromagnetic nanostructures," *npj Comput. Mater.*, vol. 6, Art. no. 12, 2020.

[30] R. Niu, L. Si, W. Zhu, X. Bao, K. Zhu, H. Sun, and F. Yang, "Hybrid neural network-driven inverse design of metasurface for ultrabroadband RCS reduction," *IEEE Trans. Antennas Propag.*, vol. 74, no. 1, pp. 837–847, Jan. 2026.

[31] C. Yu, K. Yang, X. Wu, Y. Wang, Q. Chen, W. Chen, and N. Ma, "Hybrid VAE-PSO strategy for rapid and reliable inverse design of square-loop FSS," *IEEE Antennas Wireless Propag. Lett.*, early access, 2026, doi: 10.1109/LAWP.2026.3691960.

[32] R. Zhu, T. Qiu, J. Wang, S. Sui, C. Hao, T. Liu, Y. Li, M. Feng, A. Zhang, C.-W. Qiu, and S. Qu, "Phase-to-pattern inverse design paradigm for fast realization of functional metasurfaces via transfer learning," *Nat. Commun.*, vol. 12, Art. no. 2974, 2021.

[33] J. Zhang, J. Yuan, C. Li, and B. Li, "An inverse design framework for isotropic metasurfaces based on representation learning," *Electronics*, vol. 11, no. 12, Art. no. 1844, 2022.

[34] A. Gupta, E. A. Karahan, C. Bhat, K. Sengupta, and U. K. Khankhoje, "Tandem neural network based design of multiband antennas," *IEEE Trans. Antennas Propag.*, vol. 71, no. 8, pp. 6308–6317, Aug. 2023.

[35] A. Gupta and U. K. Khankhoje, "Transfer learning based rapid design of frequency and dielectric agile antennas," *IEEE J. Multiscale Multiphys. Comput. Techn.*, vol. 10, pp. 47–57, 2024.

[36] S. Chen, G.-H. Sun, and K. Wang, "Inverse design of microstrip antennas based on deep learning," *Electronics*, vol. 14, no. 13, Art. no. 2510, 2025.

[37] M. Aoun and A. Hussain, "Automated optimization of terahertz nanoantenna design using a VAE–CNN reinforcement learning framework," *Opt. Quantum Electron.*, vol. 58, Art. no. 147, 2026.

[38] Y. Tang, K. Kojima, T. Koike-Akino, Y. Wang, P. Wu, Y. Xie, M. H. Tahersima, D. K. Jha, K. Parsons, and M. Qi, "Generative deep learning model for inverse design of integrated nanophotonic devices," *Laser Photon. Rev.*, vol. 14, no. 12, Art. no. 2000287, 2020.

[39] C. Niu, M. Phaneuf, and P. Mojabi, "A diffusion model for multi-layered metasurface unit cell synthesis," *IEEE Open J. Antennas Propag.*, vol. 4, pp. 654–666, 2023.

[40] Y. Lyu, Y. Hong, and Z. N. Chen, "Generative deep learning-enabled multiobjective synthesis of dual-polarized wide-angle loop-family frequency-selective surfaces," *IEEE Trans. Antennas Propag.*, vol. 74, no. 1, pp. 456–466, Jan. 2026.

[41] E. Khoram, Z. Wu, Y. Qu, M. Zhou, and Z. Yu, "Graph neural networks for metasurface modeling," *ACS Photon.*, vol. 10, pp. 892–899, 2023.

[42] Q. Bao, D. Zhang, X. Liu, T. Ma, and J.-J. Xiao, "GAT-Net: Inverse design of multifunctional metasurface based on graph attention network," *Opt. Laser Technol.*, vol. 183, Art. no. 112273, 2025.

[43] T. Zhang, C. Y. Kee, Y. S. Ang, E. Li, and L. K. Ang, "Symmetry enhanced network architecture search for complex metasurface design," *IEEE Access*, vol. 10, pp. 73533–73547, 2022.

[44] G. Yang, Q. Xiao, Z. Zhang, Z. Yu, X. Wang, and Q. Lu, "Exploring AI in metasurface structures with forward and inverse design," *iScience*, vol. 28, Art. no. 111995, 2025.

[45] Z. N. Chen, P. Liu, Y. Kuang, Y. Lyu, B. Zhang, D. Yang, Y. He, and J.-M. Jin, "Deep learning-enabled generative antenna design: Review and prospects," *IEEE Trans. Antennas Propag.*, vol. 74, no. 5, pp. 4051–4066, May 2026.

[46] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[47] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.

[48] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.

[49] G. Li, M. Müller, A. Thabet, and B. Ghanem, "DeepGCNs: Can GCNs go as deep as CNNs?," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9266–9275.

[50] V. Lucarini, J. J. Saarinen, K.-E. Peiponen, and E. M. Vartiainen, *Kramers–Kronig Relations in Optical Materials Research*. Berlin, Germany: Springer, 2005.

[51] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[52] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[53] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.

[54] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "β-VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[55] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," in *Proc. SIGNLL Conf. Comput. Natural Lang. Learn. (CoNLL)*, 2016, pp. 10–21.

[56] A. A. Alemi, B. Poole, I. Fischer, J. V. Dillon, R. A. Saurous, and K. Murphy, "Fixing a broken ELBO," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 159–168.

[57] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Trans. Evol. Comput.*, vol. 3, no. 4, pp. 257–271, Nov. 1999.

[58] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, "Handling multiple objectives with particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 256–279, Jun. 2004.

[59] J. Shao, Y. Lu, Y. Sun, and L. Zhao, "An improved multi-objective particle swarm optimization algorithm for the design of foundation pit of rail transit upper cover project," *Sci. Rep.*, vol. 15, Art. no. 10403, 2025.